AXIS_DATA_WIDTH_CONVERTER

AIR FORCE RESEARCH LABORATORY

March 25, 2026

Jay Convertino

Contents

1 Usage **2**
- 1.1 Introduction . . . 2
- 1.2 Dependencies . . . 2
  - 1.2.1 fusesoc_info Depenecies . . . 2
- 1.3 In a Project . . . 2

2 Architecture **2**
- 2.1 Waveforms . . . 3

3 Building **3**
- 3.1 fusesoc . . . 4
- 3.2 Source Files . . . 4
  - 3.2.1 fusesoc_info File List . . . 4
- 3.3 Targets . . . 4
  - 3.3.1 fusesoc_info Targets . . . 4
- 3.4 Directory Guide . . . 5

4 Simulation **6**
- 4.1 iverilog . . . 6
- 4.2 cocotb . . . 6

5 Module Documentation **7**
- 5.1 axis_data_width_converter . . . 8
- 5.2 tb_axis . . . 11
- 5.3 tb_cocotb verilog . . . 13
- 5.4 tb_cocotb python . . . 16

1 Usage

1.1 Introduction

This data width converter is for any size slave to master convertion. Meaning, uneven divides and even divides will be converted without issue. If flush mode is not used the core will hold data if there is not enough for the output. Flush mode will push out any left over data if TLAST is also valid.

1.2 Dependencies

The following are the dependencies of the cores.

- fusesoc 2.X
- iverilog (simulation)
- cocotb (simulation)

1.2.1 fusesoc_info Depenecies

- dep
  - AFRL:utility:helper:1.0.0
- dep_tb
  - AFRL:simulation:axis_stimulator
  - AFRL:simulation:clock_stimulator
  - AFRL:utility:sim_helper

1.3 In a Project

Simply use this core between a sink and source AXIS devices. This will convert from one BUS size to another. Check the code to see if others will work correctly.

2 Architecture

The only module is the axis_data_width_converter module. It is listed below.

- **axis_data_width_converter** Implement an algorithm to convert BUS data interfaces in even multiples (see core for documentation).

The data width converter uses a generate block to select between two possible scenarios. First is they are equal, which just assigns the fields to each other. The second is any difference in size. This will convert from one to another using a register build up method between the slave and master. If FLUSH is set and the sizes are different the core will flush out any bytes in the register when TLAST is valid.

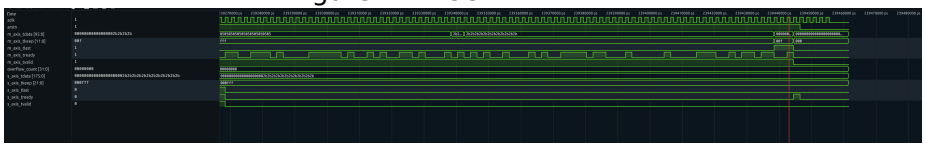
1. Create registers used to buffer data and signals
   (a) **READY** Take input data and output data while increasing or decreasing register count.
   (b) **FULL** Register is full of data and needs to have data removed before going back to ready.
   (c) **FLUSH** If the FLUSH_LAST option is set, when TLAST is found in the ready state all valid tkeep data will be flushed from the core.
   (d) **OVERFLOW** This indicates a bug in the core, clear all registers and increment overflow counter by 1 for each incident.

2.1 Waveforms

The idealized simulation waveform is shown below. The values reflect the results of using the icarus backend with surfer view tools.

Random input ready and FLUSH is set so last few bytes are pushed out and aligned.

Figure 1: FLUSH ENABLED



Please see 5 for more information.

3 Building

The AXIS data width converter core is written in Verilog 2001. They should synthesize in any modern FPGA software. The core comes as a fusesoc packaged core and can be included in any other core. Be sure to make sure you have meet the dependencies listed in the previous section. Linting is performed by verible using the lint target.

3.1 fusesoc

Fusesoc is a system for building FPGA software without relying on the internal project management of the tool. Avoiding vendor lock in to Vivado or Quartus. These cores, when included in a project, can be easily integrated and targets created based upon the end developer needs. The core by itself is not a part of a system and should be integrated into a fusesoc based system. Simulations are setup to use fusesoc and are a part of its targets.

3.2 Source Files

3.2.1 fusesoc_info File List

- src
  - src/axis_data_width_converter.v
- tb
  - 'tb/tb_axis.v': 'file_type': 'verilogSource'
- tb_cocotb
  - 'tb/tb_cocotb.py': 'file_type': 'user', 'copyto': '.'
  - 'tb/tb_cocotb.v': 'file_type': 'verilogSource'

3.3 Targets

3.3.1 fusesoc_info Targets

- default

  Info: Default for IP intergration.
- lint

  Info: Lint with Verible
- sim

  Info: Test 1:1 conversion.
- sim_reduce

  Info: Test data reduction.
- sim_rand_data_reduce

  Info: Test data reduction with random data

- sim_rand_ready_rand_data_reduce

  Info: Test data reduction with random ready and random data.

- sim_8bit_count_data_reduce

  Info: Test data reduction with counter data.

- sim_rand_ready_8bit_count_data_reduce

  Info: Test data reduction with counter data, and random ready.

- sim_increase

  Info: Test data increase.

- sim_rand_data_increase

  Info: Test data increase with random data.

- sim_rand_ready_rand_data_increase

  Info: Test data increase with random data, and random ready.

- sim_8bit_count_data_increase

  Info: Test data increase with count data.

- sim_rand_ready_8bit_count_data_increase

  Info: Test data increase with count data, and random ready.

- sim_cocotb

  Info: Cocotb unit tests

3.4 Directory Guide

Below highlights important folders from the root of the directory.

1. **docs** Contains all documentation related to this project.
   - **manual** Contains user manual and github page that are generated from the latex sources.
2. **src** Contains source files for the core
3. **tb** Contains test bench files for iverilog and cocotb

4 Simulation

There are a few different simulations that can be run for this core. All currently use iVerilog (icarus) to run. The first is iverilog, which uses verilog only for the simulations. The other is cocotb. This does a unit test approach to the testing and gives a list of tests that pass or fail.

4.1 iverilog

All simulation targets that do NOT have cocotb in the name use a verilog test bench with verilog stimulus components. These all read in a file and then write a file that has been processed by the data width converter. Then the input and output file are compared with a MD5 sum to check that they match. If they do not match then the test has failed. All of these tests provide fst output files for viewing the waveform in the there target build folder.

4.2 cocotb

To use the cocotb tests you must install the following python libraries.

```
$ pip install cocotb
$ pip install cocotbext-axi
```

Then you must use the cocotb sim target. In this case it is sim_cocotb. This target can be run with various bus and fifo parameters.

```
$ fusesoc run --target sim_cocotb AFRL:
    ↪ streaming_converter:axis_data_width_converter
    ↪ :2.0.0 --SLAVE_WIDTH=7 --MASTER_WIDTH=32
```

The following is an example command to run through various parameters without typing them one by one.

```
$ echo "STARTING␣TEST"; for i in {1..32}; do sleep 5;
    ↪ export RY=$((RANDOM%31+1)); fusesoc run --target
    ↪ sim_cocotb AFRL:streaming_converter:
    ↪ axis_data_width_converter:2.0.0 --SLAVE_WIDTH=$i
    ↪ --MASTER_WIDTH=$RY; echo "SLAVE␣WIDTH:" $i "
    ↪ MASTER␣WIDTH:" $RY; done
```

5 Module Documentation

Natural docs is used to generate documentation for this project. The next lists the following sections.

- **axis_data_width_converter** AXIS data width converter, converts from one BUS data size to another.

- **tb_axis** Verilog test bench.

- **tb_cocotb verilog** Verilog test bench base for cocotb.

- **tb_cocotb python** cocotb unit test functions.

axis_data_width_converter.v

AUTHORS

JAY CONVERTINO

DATES

2026/03/24

INFORMATION

Brief

AXIS DATA WIDTH CONVERTER

License MIT

Copyright 2026 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

axis_data_width_converter

```
module axis_data_width_converter #(
parameter
SLAVE_WIDTH
 =
1,
parameter
MASTER_WIDTH
 =
1,
parameter
FLUSH_LAST
 =
1
)
```

```
(
input
wire
aclk,
input
wire
arstn,
output
wire
 [(MASTER_WIDTH*8)-1:0]
m_axis_tdata,
output
wire
 [MASTER_WIDTH-1:0]
m_axis_tkeep,
output
wire
m_axis_tvalid,
input
wire
m_axis_tready,
output
wire
m_axis_tlast,
input
wire
 [(SLAVE_WIDTH*8)-1:0]
s_axis_tdata,
input
wire
 [SLAVE_WIDTH-1:0]
s_axis_tkeep,
input
wire
s_axis_tvalid,
output
wire
s_axis_tready,
input
wire
s_axis_tlast,
output
wire
 [31:0]
overflow_count
)
```

Change size of streaming bus in even integers of. 1/2 2/1 2/4 4/2 etc.

Parameters

SLAVE_WIDTH parameter	Width of the slave input bus in bytes
MASTER_WIDTH parameter	Width of the master output bus in bytes
FLUSH_LAST parameter	Once last bytes are received, force them out regardless of size (no longer ready to accept more data once this happens).

Ports

aclk input wire	Clock for AXIS
arstn input wire	Negative reset for AXIS

m_axis_tdata `output wire [(MASTER_WIDTH* 8)- 1:0]`	Output data
m_axis_tvalid `output wire`	When active high the output data is valid
m_axis_tready `input wire`	When set active high the output device is ready for data.
m_axis_tlast `output wire`	Indicates last word in stream.
s_axis_tdata `input wire [(SLAVE_WIDTH* 8)- 1:0]`	Input data
s_axis_tvalid `input wire`	When set active high the input data is valid
s_axis_tready `output wire`	When active high the device is ready for input data.
s_axis_tlast `input wire`	Is this the last word in the stream (active high).
overflow_count `output wire [31:0]`	How many times has the core overflowed. This is a indicator of a bug in the code or the core. If this happens, data is lost due to state reset.

STATES

Core has 4 states, including an overflow(error) state.

<READY>	d1
<FULL>	d2
<FLUSH>	d3
<OVERFLOW>	d0

tb_axis.v

AUTHORS

JAY CONVERTINO

DATES

2024/12/09

INFORMATION

Brief

Test bench for axis_data_width_converter using axis stim and clock stim.

License MIT

Copyright 2024 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

tb_axis

```
module tb_axis #(
parameter
IN_FILE_NAME
 =
in.bin,
parameter
OUT_FILE_NAME
 =
out.bin,
parameter
RAND_READY
 =
0,
parameter
```

```
 SLAVE_WIDTH
  =
 2,
 parameter
 MASTER_WIDTH
  =
 4
)
                                                                                (
 )
```

Test bench for axis_data_width_converter. This will run a file through the system and write its output. These can then be compared to check for errors. If the files are identical, no errors. A FST file will be written.

Parameters

IN_FILE_NAME parameter	File name for input.
OUT_FILE_NAME parameter	File name for output.
RAND_READY parameter	0 = no random ready. 1 = randomize ready.
SLAVE_WIDTH parameter	Width of the slave input bus in bytes
MASTER_WIDTH parameter	Width of the master output bus in bytes

INSTANTIATED MODULES

clk_stim

Generate a 50/50 duty cycle set of clocks and reset.

slave_axis_stim

Device under test SLAVE stimulus module.

dut

Device under test, axis_data_width_converter

master_axis_stim

Devie under test MASTER stimulus module.

tb_cocotb.v

AUTHORS

JAY CONVERTINO

DATES

2024/12/11

INFORMATION

Brief

Test bench wrapper for cocotb

License MIT

Copyright 2024 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

tb_cocotb

```
module tb_cocotb #(
parameter
SLAVE_WIDTH
 =
1,
parameter
MASTER_WIDTH
 =
1
)
                                                                    (
input
wire
aclk,
```

```
 input
 wire
 arstn,
 output
 wire
  [(MASTER_WIDTH*8)-1:0]
 m_axis_tdata,
 output
 wire
  [MASTER_WIDTH-1:0]
 m_axis_tkeep,
 output
 wire
 m_axis_tvalid,
 input
 wire
 m_axis_tready,
 output
 wire
 m_axis_tlast,
 input
 wire
  [(SLAVE_WIDTH*8)-1:0]
 s_axis_tdata,
 input
 wire
  [SLAVE_WIDTH-1:0]
 s_axis_tkeep,
 input
 wire
 s_axis_tvalid,
 output
 wire
 s_axis_tready,
 input
 wire
 s_axis_tlast
)
```

Test bench for data width converter. This will run a file through the system and write its output. These can then be compared to check for errors. If the files are identical, no errors. A FST file will be written.

Parameters

SLAVE_WIDTH parameter	Width of the slave input bus in bytes
MASTER_WIDTH parameter	Width of the master output bus in bytes
REVERSE	Change byte order

Ports

aclk input wire	Clock for AXIS
arstn input wire	Negative reset for AXIS
m_axis_tdata output wire [(MASTER_WIDTH* 8)- 1:0]	Output data
m_axis_tvalid output wire	When active high the output data is valid
m_axis_tready input wire	When set active high the output device is ready for data.

m_axis_tlast `output wire`	Indicates last word in stream.
s_axis_tdata `input wire [(SLAVE_WIDTH* 8)- 1:0]`	Input data
s_axis_tvalid `input wire`	When set active high the input data is valid
s_axis_tready `output wire`	When active high the device is ready for input data.
s_axis_tlast `input wire`	Is this the last word in the stream (active high).

INSTANTIATED MODULES

dut

Device under test, axis_data_width_converter

tb_cocotb.py

AUTHORS

JAY CONVERTINO

DATES

2024/12/09

INFORMATION

Brief

Cocotb test bench

License MIT

Copyright 2024 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

FUNCTIONS

random_bool

```
def random_bool()
```

Return a infinte cycle of random bools

Returns: List

start_clock

```
def start_clock(
dut
)
```

Start the simulation clock generator.

Parameters

dut Device under test passed from cocotb test function

reset_dut

```
async def reset_dut(
dut
)
```

Cocotb coroutine for resets, used with await to make sure system is reset.

Parameters

dut Device under test passed from cocotb.

conversion_test

```
@cocotb.test()
async def conversion_test(
dut
)
```

Coroutine that is identified as a test routine. This routine tests for conversion based on current input to output size conversion.

Parameters

dut Device under test passed from cocotb.

conversion_test_random_ready

```
@cocotb.test()
async def conversion_test_random_ready(
dut
)
```

Coroutine that is identified as a test routine. This routine tests for randomized ready from the sink.

Parameters

dut Device under test passed from cocotb.

conversion_test_flush_last

```
@cocotb.test()
async def conversion_test_flush_last(
dut
```

```
)
```

Coroutine that is identified as a test routine. This routine tests for randomized ready from the sink.

Parameters

dut Device under test passed from cocotb.

conversion_test_flush_last_random_ready

```
@cocotb.test()
async def conversion_test_flush_last_random_ready(
dut
)
```

Coroutine that is identified as a test routine. This routine tests for randomized ready from the sink.

Parameters

dut Device under test passed from cocotb.

in_reset

```
@cocotb.test()
async def in_reset(
dut
)
```

Coroutine that is identified as a test routine. This routine tests if device stays in unready state when in reset.

Parameters

dut Device under test passed from cocotb.

no_clock

```
@cocotb.test()
async def no_clock(
dut
)
```

Coroutine that is identified as a test routine. This routine tests if no ready when clock is lost and device is left in reset.

Parameters

dut Device under test passed from cocotb.